```
# ======================================================================
# CELL 1: DISCOVERY PHASE - DATA CLEANING & BINARY LABELING
# ======================================================================
import pandas as pd
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import TextVectorization, Embedding, GlobalAveragePooling1D, Dense
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# 1. Simulate or load an e-commerce review dataset
# In your local setup, swap this block for: df = pd.read_csv('your_reviews.csv')
np.random.seed(42)
mock_reviews = [
    "I absolutely love this product! Highly recommended.", "Fantastic quality, arrived quickly.",
    "The product arrived broken and I am very unhappy", "Terrible customer service and cheap material.
    "It is okay, nothing special but works fine.", "Worst purchase I have ever made. Dissatisfied.",
    "Surprisingly good for the price. Happy customer!", "Waste of money. Stopped working after a day."
] * 50
mock_ratings = [5, 5, 1, 1, 3, 1, 4, 1] * 50
df = pd.DataFrame({'review_text': mock_reviews, 'rating': mock_ratings})

print(f"Initial Dataset Shape: {df.shape}")

# 2. Handle missing values explicitly
df['review_text'] = df['review_text'].fillna("").astype(str)

# 3. Apply strict binary labeling and drop neutral 3-star reviews to sharpen boundaries
df = df[df['rating'] != 3].reset_index(drop=True)
df['sentiment'] = df['rating'].apply(lambda x: 1 if x >= 4 else 0)

# 4. Isolate features and targets
X = df['review_text'].values
y = df['sentiment'].values

# 5. Stratified Train/Test Split (80/20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(f"Train samples: {len(X_train)} | Test samples: {len(X_test)}")
```

```
Initial Dataset Shape: (400, 2)
Train samples: 280 | Test samples: 70
```

```
# ======================================================================
# CELL 2: TECHNICAL PHASE - TEXT VECTORIZATION & MODEL BUILDING
# ======================================================================
VOCAB_SIZE = 10000
MAX_SEQUENCE_LENGTH = 100
EMBEDDING_DIM = 16

# 1. Initialize and adapt the Keras TextVectorization layer
vectorizer = TextVectorization(
    max_tokens=VOCAB_SIZE,
    output_mode='int',
    output_sequence_length=MAX_SEQUENCE_LENGTH,
    standardize='lower_and_strip_punctuation'
)

# Build the internal vocabulary index from the training corpus strings
vectorizer.adapt(X_train)
print(f"Vocabulary successfully built. Top 10 words: {vectorizer.get_vocabulary()[:10]}")

# 2. Vectorize the Train and Test splits to numeric format to comply with Keras 3 inputs
X_train_vectorized = vectorizer(X_train)
X_test_vectorized = vectorizer(X_test)

# 3. Build the Neural Network Architecture expecting numerical array tensors
```

```
model = Sequential([
    Embedding(input_dim=VOCAB_SIZE, output_dim=EMBEDDING_DIM, name='Embedding'),
    GlobalAveragePooling1D(name='Global_Average_Pooling'),
    Dense(16, activation='relu', name='Hidden_Dense_Layer'),
    Dense(1, activation='sigmoid', name='Output_Neuron')
])

model.summary()

# 4. Compile the model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['binary_accuracy']
)

# 5. Train the model using the vectorized matrices
print("\nCommencing training loop...")
history = model.fit(
    X_train_vectorized, y_train,
    epochs=10,
    batch_size=16,
    validation_data=(X_test_vectorized, y_test),
    verbose=1
)
```

```
Vocabulary successfully built. Top 10 words: ['', '[UNK]', np.str_('i'), np.str_('product'), np.str_('
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
Embedding (Embedding)	?	0 (unbuilt)
Global_Average_Pooling (GlobalAveragePooling1D)	?	0
Hidden_Dense_Layer (Dense)	?	0 (unbuilt)
Output_Neuron (Dense)	?	0 (unbuilt)

```
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)

Commencing training loop...
Epoch 1/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 2s 20ms/step - binary_accuracy: 0.5714 - loss: 0.6862 - val_binary_accuracy
Epoch 2/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - binary_accuracy: 0.5714 - loss: 0.6826 - val_binary_accuracy:
Epoch 3/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - binary_accuracy: 0.5714 - loss: 0.6793 - val_binary_accuracy:
Epoch 4/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - binary_accuracy: 0.5714 - loss: 0.6781 - val_binary_accuracy:
Epoch 5/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - binary_accuracy: 0.5714 - loss: 0.6749 - val_binary_accuracy
Epoch 6/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - binary_accuracy: 0.5714 - loss: 0.6720 - val_binary_accuracy
Epoch 7/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - binary_accuracy: 0.5714 - loss: 0.6691 - val_binary_accuracy
Epoch 8/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - binary_accuracy: 0.5714 - loss: 0.6627 - val_binary_accuracy
Epoch 9/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - binary_accuracy: 0.5714 - loss: 0.6562 - val_binary_accuracy
Epoch 10/10
18/18 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - binary_accuracy: 0.5714 - loss: 0.6457 - val_binary_accuracy
```

```
# ======================================================================
# CELL 3: ACTION PHASE - CRITICAL TEST STRINGS & BUSINESS ROUTING
# ======================================================================
# 1. Build raw input string array
test_review = np.array(["The product arrived broken and I am very unhappy"])
```

```
# 2. Convert raw text to tokenized sequences using the fitted vectorizer
test_review_vectorized = vectorizer(test_review)

# 3. Generate stable mathematical prediction
predicted_prob = model.predict(test_review_vectorized, verbose=0)[0][0]

print("--- MANDATORY TEST REVIEW PREDICTION ---")
print(f"Review: \"{test_review[0]}\"")
print(f"Predicted Positive Sentiment Probability: {predicted_prob:.4f}")

# 4. Hardcoded Business Automation Logic
NEGATIVITY_THRESHOLD = 0.20

print("\n--- AUTOMATED CUSTOMER WORKFLOW PROCESSOR ---")
if predicted_prob < NEGATIVITY_THRESHOLD:
    print(f"🚨 ALERT: Sentiment score ({predicted_prob:.4f}) is below threshold of {NEGATIVITY_THRESH
    print("Action Taken: Flagging review and automatically routing ticket to PRIORITY CUSTOMER SUPPOR
else:
    print(f"✅ PASS: Sentiment score ({predicted_prob:.4f}) is stable. No emergency routing needed."
```

```
--- MANDATORY TEST REVIEW PREDICTION ---
Review: "The product arrived broken and I am very unhappy"
Predicted Positive Sentiment Probability: 0.4122

--- AUTOMATED CUSTOMER WORKFLOW PROCESSOR ---
✅ PASS: Sentiment score (0.4122) is stable. No emergency routing needed.
```